## Before start, make sure you have read the following articles:

- Anatomy of Channels in Go
- How to Avoid Race Conditions in Go
- Goroutines and Channels Mechanics
- Critical Pitfails Every Developer Must Avoid
- Golang Concurrency Interview Questions and Answers

## Step 1: An easy introduction

### Let's start with a simple example:

```go
func goroutineHello() {
    fmt.Println("goroutine hello")
}

func main() {
    fmt.Println("main hello")

    go goroutineHello()
}
```

> **Let's run it!**
>
> ```
> $ go run first.go
> main hello
> ```

> **Hey, where's the goroutine output?**
>
> **Why don't we see "goroutine hello"?**

> **Try to find the answer yourself!**

---

**To solve this, we need a SYNCHRONIZATION mechanism that tells the main function to WAIT for the goroutine we created to finish executing before ending the program's execution. This is where waitGroup come in (literally waiting groups) in Go.**

---

### Think of it as a counter (it's almost literally that):

- It starts at 0
- While it's greater than 0, we tell the main function to wait before continuing its execution and ending the program
- When it returns to 0, the program can finish

---

> ### Let's see how to implement it:

```go
func goroutineHello(wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Println("goroutine hello")
}

func main() {
    var wg sync.WaitGroup
    fmt.Println("main hello")

    wg.Add(1)
    go goroutineHello(&wg)
    wg.Wait()
}
```

```
$ go run first.go

main hello
goroutine hello
```

---

### Ok, let's understand what each waitGroup operation does:

### - wg.Add(1) adds 1 to the counter, to signal that we're creating a goroutine

### - wg.Done() decreases the counter by 1, signaling that the goroutine has finished

### - wg.Wait() asks the main function to wait until this counter reaches zero

> What if we call wg.Done() when the counter is zero?
>
> Why we used wg parameter as a pointer?

### Now let’s make it a bit more complicated — how about creating several goroutines at once?

```
func goroutine(num int, wg *sync.WaitGroup) {
     defer wg.Done()
     fmt.Printf("goroutine: %d\n", num)
}

func main() {
     var wg sync.WaitGroup
     fmt.Println("hello from main")

     for i := 0; i < 10; i++ {
          wg.Add(1)
          go goroutine(i, &wg)
     }
     wg.Wait()
}
```

> **Something interesting happens when we run this code several times. Run it yourself!**
>
> **Why is this happening?**

---

## Let's take control over it!

### What if we want the goroutines to execute in sequence? For this, we’ll use channels — a special Go feature that allows passing values between goroutines.

```
func goroutineCh(num int, wg *sync.WaitGroup, ch chan bool) {
    defer wg.Done()
    fmt.Println("Waiting for permission to start...")
    <-ch
    fmt.Printf("Goroutine: %d\n", num)
    ch <- true
}

func main() {
    var wg sync.WaitGroup

    ch := make(chan bool, 1)
    ch <- true
    fmt.Println("hello world from main")

    for i := 0; i < 10; i++ {
```

```
        wg.Add(1)
        go goroutineCh(i, &wg, ch)
        ch <- true
        fmt.Println("waiting for previous goroutine to finish...")
        <-ch
    }
    wg.Wait()
}
```

> **With this, we get the desired output.**

## Conclusion for part 1:

- **Use go to create goroutines**
- **Use waitGroup to synchronize with the main function**
- **Pass waitGroup by reference**
- **Channels (chan < `type` >) help control flow between goroutines**
- **Go's scheduler is non-deterministic by default**

---

# Individual Assignment (3 points):

## Solve the following problems:

### 1. Problem: Thread-Safe maps

**Imagine a situation where a lot of users of your website attempting to write a different number to the same map using the key "key".**

```
package main

import (
    "fmt"
)

func main() {
    unsafeMap := make(map[string]int)

    for i := 0; i < 100; i++ {
        go func(key int) {
            unsafeMap["key"] = key
        }(i)
    }

    value := unsafeMap["key"]
    fmt.Printf("Value: %d\n", value)
}
```

## Your Task:

> Provide 2 different ways to make the map interaction thread-safe using:

- sync.Map instead of regular map

- sync.RWMutex with a regular map

> Demonstrate the execution of the code using **-race** flag.

## Supporting Materials:

- Golang official documentation: RWMutex

- Golang official documentation: sync.Map

---

## 2. Problem: Concurrent Counter

You are developing analytics service for your company that counts total requests from users to your website. For maximum performance, each request processes in a separate goroutine and increments the global counter by 1.

Below is the legacy code from a previous vibe-coder colleague:

```go
func main() {
    var counter int

    var wg sync.WaitGroup

    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++
        }()
    }

    wg.Wait()

    fmt.Println(counter)
}
```

> Run this code several times. You will notice something strange...

## Your task:

1. Write 1-sentence technical-style explanation for your teamlead, describing why the final answer is not 1000.

2. Use built-in Go tool for detecting such problems. Provide the result of it:

```
go run -race {filename}
```

3. Provide 2 different ways to fix this problem(without using channels) to guarantee your program returns exactly 1000. Provide 2 versions of code.

Supporting materials:

- Tour of Go: sync/mutex
- sync/atomic official documentation

---

## 3. Fan-In Pattern: Metrics Aggregation System

You are a golang department architect and developing monitoring system. You have 3 servers, each sending data(metrics) in real-time. Collect data from all the sources to a centralized stream to save the data to database in the future.

## Source Code:

```go
// Stage 1
func startServer(ctx context.Context, name string) <-chan string {
	out := make(chan string)
	go func() {
		defer close(out)
		for {
			select {
			case <-ctx.Done():
				return
			case <-time.After(time.Duration(rand.Intn(500)) *
time.Millisecond):
				out <- fmt.Sprintf("[%s] metric: %d", name,
rand.Intn(100))
			}
		}
	}()
	return out
}
```

## Your Task:

### 1. Implement FanIn function:

- The function should accept slice of channels ... <-chan string as a parameter. Channels are created via `startServer` func. It is not allowed to edit `startServer` function.

- The function should merge all incoming metric messages to a single result channel.

- Important note: Use `sync.WaitGroup` to guarantee the result channel will close only when all incoming channels are closed.

- The function must also support completion via `context.Context`.

### 2. Write `main`:

- Initialize a `context` with `cancel` (or timeout of 2 seconds)

- Start `3` servers named `Alpha`, `Beta`, and `Gamma`.

- Pass these channels to your `FanIn` func

- Read data from result channel using `for range` and display it to terminal.

### Example `main`:

```go
func main() {
    ctx, cancel := context.WithTimeout(context.Background(),
2*time.Second)
    defer cancel()
    ch1 := startServer(ctx, "Alpha")
    ch2 := startServer(ctx, "Beta")
    ch3 := startServer(ctx, "Gamma")
    ch4 := FanIn(ctx, ch1, ch2, ch3)

    for val := range ch4 {
        fmt.Println(val)
    }
}
```

### Example `result`:

```
$ go run -race pipeline.go
[Alpha] metric: 20

[Beta] metric: 43
```

```
[Gamma] metric: 73

[Beta] metric: 80

[Gamma] metric: 84

[Alpha] metric: 64

...
```

---

## Demo video scenario:

- Run all of your solutions using `-race` flag and demonstrate the result.

## Definition of Done:

- All of the solutions are correct according to the Golang official documentation
- Demo video is recorded according to the `Demo video scenario`

## Deliverables:

- Push all the changes to your GitHub repository
- Submit the link to this repository(Please, submit the link as a LINK ATTACHMENT, not a file)
- Submit [1-2]-minute demo video of demonstrating your solutions as a VIDEO-FILE ATTACHMENT. You can speed it up if it is too long.

## > Below are marking criteria:

| Criteria | Weight |
|---|---|
| Problem 1 | 0.75 points |
| Problem 2 | 0.75 points |
| Problem 3 | 1.5 points |
| OVERALL | 3 points |

## > Below are penalties criteria:

**Adherence to all the requirements above is mandatory. Failure to implement any of the specified points (e.g., missing `sync.WaitGroup` or neglecting `context` support) will result in a significant point deduction.**

| Category | Criteria | Penalty |
|---|---|---|
| Video Demo | Video Demo is does not Present or Unable to Open/View or Contradicts ethical standards | 3 points |
| Video Demo Link | The video demo is specified in any way other than the `video-file` attachment | 1.5 points |
| Git Repo Link | The link to the git repository is specified in any way other than the `link-attachment` | 1.5 points |
| Git Repo Content | The content of git repository for this practice is corrupted or unable to Open/Read | 3 points |
| Your Task part | At least one requirement (e.g. Demonstrate the execution of the code using -race flag.) from `Your Task` part is failed to meet | 100% of a problem's solution mark |
| Cheating/ Malpractice | Students practicing cheating, duplicating the contents of other students solutions | 100% of the `Overall` mark for each of the students |

---

> ## DEADLINE: 29.03.2026, SUNDAY 23:59

---

> ## OPTIONAL FEATURES (Choose Freely)

> **Students may implement any number of the following to increase difficulty and engagement.**

### 🧙 ADVANCED: "The Gopher Wizard"

- **Concurrency Patterns:**
  **Implement any number of Concurrency Patterns.**
  **Example(click)**